

Mobile App Architecture

App: GuardTrackingApp (TracSOpro)

Framework: React Native 0.82 + React 19

Last updated: June 2026

Overview

The mobile app serves four user roles — Guard, Client, Admin, and Super Admin — from a single codebase. Role-based navigation renders different tab stacks after authentication. State is managed with Redux Toolkit; only the auth slice is persisted to AsyncStorage.

```
App.tsx
├─ ErrorBoundary
├─ SafeAreaProvider
├─ ThemeProvider
├─ Redux Provider + PersistGate (auth only)
└─ AppNavigator
    ├─ Splash (rehydration gate)
    ├─ AuthNavigator (unauthenticated)
    └─ MainNavigator (authenticated, role-based)
```

Directory structure

```
GuardTrackingApp/src/
├─ navigation/           # AppNavigator, role navigators, tab bars
├─ screens/              # UI screens grouped by role and feature
│   ├─ auth/             # Login, signup, OTP, password reset
│   ├─ dashboard/        # Guard home, shifts, reports
│   ├─ guard/            # Check-in/out, shift details
│   ├─ client/           # Sites, guards, billing
│   ├─ admin/            # Operations, user management
│   ├─ superAdmin/       # Platform management
│   ├─ chat/             # Messaging (shared)
│   ├─ notifications/
│   ├─ settings/
│   └─ support/
├─ components/           # Reusable UI (common, client, admin, shift, ui)
├─ services/             # API, WebSocket, location, push, payments
├─ store/                # Redux slices and store config
├─ hooks/                # Custom hooks (location, subscription limits)
├─ config/               # apiConfig.ts – API and WebSocket URLs
├─ types/                # TypeScript definitions
└─ utils/                # Theme, logger, helpers
```

Navigation

Top-level flow

AppNavigator checks auth state from Redux:

1. **Loading** — Splash screen while PersistGate rehydrates auth.
2. **Unauthenticated** — **AuthNavigator** (onboarding, login, signup flows).
3. **Authenticated** — **MainNavigator** routes by **user.role**.

Role routing (**MainNavigator.tsx**)

Role	Navigator	Tab count
SUPER_ADMIN	SuperAdminNavigator	5 tabs
ADMIN	AdminNavigator	5 tabs
CLIENT	ClientStackNavigator	5 tabs
GUARD (default)	GuardStackNavigator → DashboardNavigator	6 tabs

ImpersonationBanner renders above all role navigators when a super admin is impersonating a user.

Guard tabs (**DashboardNavigator**)

Tab	Screen	Purpose
Home	GuardHomeScreen	Dashboard, active shift summary
Check In/Out	CheckInScreen	Quick check-in/out
My Shifts	MyShiftsScreen	Shift list and details
Reports	ReportsScreen	Incident reports
Chat	Chat screens	Messaging
Settings	GuardSettingsScreen	Profile and preferences

Client tabs

Dashboard, Sites & Shifts, Reports, Guards, Settings — with stack screens for site creation, shift scheduling, guard details, and Stripe billing.

Admin tabs

Dashboard, Operations Center, Management (users/invitations/sites/shifts), Reports (incidents/analytics), Settings (subscription, system).

Super Admin tabs

Dashboard, Companies, Analytics, Billing, Settings — with impersonation and audit log screens.

State management

Redux store (`store/index.ts`)

Slice	State domain	Persisted
<code>auth</code>	User, tokens, impersonation	Yes
<code>shifts</code>	Shift list, active shift, check-in/out	No
<code>shiftReports</code>	Guard shift reports	No
<code>locations</code>	GPS, geofence, live locations	No
<code>client</code>	Client dashboard, sites, guards	No
<code>admin</code>	Admin dashboard stats	No
<code>guards</code>	Guard CRUD (admin view)	No
<code>incidents</code>	Legacy incidents	No
<code>emergency</code>	Emergency alerts	No
<code>notifications</code>	In-app notifications	No
<code>chat</code>	Conversations and messages	No
<code>messages</code>	Legacy messages	No

Only `auth` is whitelisted in `redux-persist` with `AsyncStorage`. Sensitive tokens also use `react-native-keychain` via the security manager.

Key async flows

- **Login** → `authSlice.login` → stores tokens → `WebSocketService.connect()`
- **Check-in** → `shiftSlice.checkIn` → GPS validation → location tracking starts
- **Location** → `locationTrackingService` → REST + WebSocket upload
- **Logout** → clears auth, disconnects WebSocket, stops location tracking

Services layer

Service	File	Responsibility
API client	<code>services/api.ts</code>	Central Axios instance, auth interceptors, token refresh
WebSocket	<code>services/WebSocketService.ts</code>	Socket.io connection, realtime events
Location tracking	<code>services/locationTrackingService.ts</code>	Background GPS polling
Location validation	<code>services/locationValidationService.ts</code>	Geofence check-in validation
Notifications	<code>services/notificationService.ts</code>	FCM registration and handlers

Service	File	Responsibility
Shifts	<code>services/shiftService.ts</code>	Shift API wrappers
Sites	<code>services/siteService.ts</code>	Client site management
Payments	<code>services/paymentService.ts</code> , <code>stripeService.ts</code>	Stripe billing
Super Admin	<code>services/superAdminService.ts</code>	Platform admin API
Support	<code>services/supportApiService.ts</code>	Support tickets
Camera	<code>services/cameraService.ts</code>	Photo/video capture for reports
Biometric	<code>services/biometricAuthService.ts</code>	Optional biometric login

API configuration (`config/apiConfig.ts`)

URLs are resolved at runtime:

Mode	API URL	When
Development	<code>http://<LAN_IP>:3000/api</code>	<code>__DEV__</code> is true
Local LAN release	Same LAN URL	<code>USE_LOCAL_LAN_RELEASE = true</code>
Production	<code>https://api.tracsopro.com/api</code>	Release build, LAN flag false

Release requirement: Set `USE_LOCAL_LAN_RELEASE = false` and verify HTTPS URLs before store submission.

API client pattern

`api.ts` attaches the Bearer token from Redux on every request. On 401, it attempts a token refresh via `/api/auth/refresh` before retrying. All domain methods (shifts, sites, chat, etc.) are centralized in this file.

Realtime integration

On successful login, `WebSocketService`:

1. Connects to `getWebSocketUrl()`.
2. Emits `authenticate` with the JWT.
3. Listens for `live_locations_update`, `emergency_alert`, `notification`, and chat events.
4. Dispatches to `locationSlice` and `notificationSlice`.

Guards emit `location_update` during active shifts. Admins and clients receive `live_locations_update` broadcasts.

Push notifications

Firebase Cloud Messaging (`@react-native-firebase/messaging`):

1. Request permission on first launch.

2. Register device token via `POST /api/notifications/register-device`.
3. Handle foreground, background, and quit-state notifications.
4. Deep-link to relevant screens based on notification `data` payload.

Requires `google-services.json` (Android) and Firebase iOS config.

Location and geofencing

1. Guard checks in → `locationValidationService` validates GPS against site `radiusMeters`.
2. `locationTrackingService` starts periodic GPS uploads during active shifts.
3. Geofence enter/exit events sent via REST and WebSocket.
4. Admin Operations Center and client dashboards display live guard positions on maps (`react-native-maps`).

Payments (Stripe)

Client and admin billing screens use `@stripe/stripe-react-native`:

- Payment method setup via Stripe SetupIntent.
- Subscription checkout via backend-generated Stripe session.
- Billing portal for invoice management.

Theming and UI

- `utils/theme.tsx` provides `ThemeProvider` with light/dark support.
- Shared components in `components/ui/` (buttons, cards, stats grid).
- Role-specific components in `components/client/`, `components/admin/`, etc.
- SVG icons in `assets/icons/`.

Error handling

- `ErrorBoundary` wraps the entire app in `App.tsx`.
- `PersistGateWithTimeout` prevents infinite loading if AsyncStorage rehydration stalls (5s timeout).
- API errors surface via Redux thunk `rejectWithValue` and screen-level alerts.

Build targets

Platform	Entry	Build command
Android	<code>android/app/src/main/java/com/tracsopro/</code>	<code>npm run android</code>
iOS	<code>ios/GuardTrackingApp/AppDelegate.swift</code>	<code>npm run ios</code>
Release APK	Gradle <code>assembleRelease</code>	<code>npm run android:release:local</code>

Testing

- Unit tests: Jest (`__tests__/App.test.tsx`).
- Lint: ESLint (`npm run lint`).
- E2E: Planned (Detox).

Related documents

- [SYSTEM_ARCHITECTURE.md](#) — end-to-end system
- [API_REFERENCE.md](#) — backend endpoints
- [DEPLOYMENT_GUIDE.md](#) — release builds
- [USER_GUIDE.md](#) — role workflows